



FUNCTIONAL AND TECHNICAL DOCUMENTATION

Smart Healthcare Platform

Intelligent clinical decision support platform

[DB] Patient data collection → [NLP] Medical NLP analysis

! Risk assessment → [MD] Doctor dashboard → (OK) Clinical review

FastAPI

React

MySQL

SQLAlchemy

medspaCy NLP

Important notice

(i)

Smart Healthcare constitutes a clinical decision support system. It does not replace the healthcare professional and should not be considered as an autonomous diagnostic system. Artificial intelligence provides analysis assistance; **the final clinical decision remains, under all circumstances, the responsibility of the healthcare professional.**

About this document.

This document constitutes the functional and technical documentation of the *Smart Healthcare* platform. It describes the overall system architecture, the details of implemented modules (authentication, patient management, multilingual NLP pipeline, risk scoring engine, dashboards, appointment management, notifications, administration), the database schema, frontend/backend communication, as well as the complete system execution scenario, from user login to final clinical review.

Contents

1	General overview	1
1.1	Project objectives	1
2	Overall architecture	2
2.1	Description of layers	2
2.1.1	Frontend	2
2.1.2	Backend	2
2.1.3	NLP engine	2
2.1.4	Risk engine	3
2.1.5	Database	3
3	Implemented features	4
3.1	Authentication and account management	4
3.2	Role and permission management	4
3.3	Patient management	5
3.3.1	Patient profile (self-service)	5
3.3.2	Direct admission (walk-in)	5
3.4	Clinical admission (<i>Patient Intake</i>)	5
3.5	Multilingual NLP pipeline	5
3.6	Risk scoring engine	5
3.7	Doctor dashboard	5
3.8	Administrator dashboard	6
3.9	Clinical notes	6
3.10	Appointment management	6
3.11	Notifications	6
3.12	Account settings	6
3.13	Support	7
4	Patient pipeline architecture	8
4.1	Pipeline steps	8
4.1.1	Admission	8
4.1.2	Validation	8
4.1.3	Storage	8
4.1.4	Analysis	8
4.1.5	Assessment	8
4.1.6	Presentation	8
4.1.7	Review	9
5	NLP architecture	10
5.1	Bilingual/multilingual architecture	10
5.2	Medical entity extraction	10
5.3	Negation detection	11
5.4	NLP technologies	11

6 Risk engine architecture	12
6.1 Principle	12
6.2 Factors considered and weighting	12
6.3 Risk levels	13
6.4 Calculation example	13
7 Database architecture	14
7.1 Entity-relationship model	14
7.2 Main entities	14
7.3 Relationships	15
8 Security and access control	16
8.1 Authentication	16
8.2 Authorization	16
8.3 Data isolation	16
9 Frontend architecture	17
9.1 Code organization	17
9.2 Routing and access protection	17
9.3 Session management	17
10 Frontend – Backend communication	18
10.1 Endpoint mapping	18
11 Technology stack	19
12 Complete usage scenario (from A to Z)	20
13 Operation example	21
13.1 Data entered	21
13.2 NLP extraction	21
13.3 Assessment	21
13.4 Interpretation	21
14 Synthetic system view	22
15 Screenshots of interfaces	23
15.1 Authentication	23
15.2 Patient Space	25
15.2.1 Dashboard and admissions	25
15.2.2 New admission (/patient-form)	25
15.2.3 Risk assessment result	28
15.2.4 Notifications	28
15.3 Doctor Space	30
15.4 Administration	31
16 Conclusion	32

General overview

CHAPTER

Smart Healthcare is an intelligent web platform designed to assist healthcare professionals in collecting, structuring, and analyzing patient clinical information.

The platform combines a web interface (React/TypeScript), a backend API (FastAPI), a relational database (MySQL), and natural language processing (NLP) techniques to transform information entered during admission into structured clinical data, usable for calculating a risk score and clinical prioritization.

The system's main objective is to provide the healthcare professional with a synthetic view of the patient accompanied by a risk level assessment and the factors contributing to this assessment, while maintaining complete traceability (clinical notes, admission history, appointments).

1.1 Project objectives

- Centralize patient information within the same clinic.
- Facilitate admission and clinical data collection, in self-service (patient) or direct admission (doctor/admin).
- Structure symptoms, medical history, treatments and vital signs.
- Use NLP to automatically identify relevant medical information, in English, French and Arabic.
- Detect negation and context of symptoms to avoid clinical false positives.
- Automatically calculate a weighted and explainable risk score.
- Identify and return the contributing risk factors.
- Present results to the doctor via a synthetic dashboard, with intelligent grouping of records.
- Allow the healthcare professional to add clinical observations (notes).
- Schedule and track appointments between patients and doctors.
- Notify users of relevant events (appointments, notes).
- Secure access to features according to roles (patient, doctor, administrator).
- Strictly isolate data between clinics (multi-tenant architecture).

Overall architecture

CHAPTER

The overall architecture of Smart Healthcare is organized into four complementary layers: user interface, business API, clinical analysis engine, and data persistence.

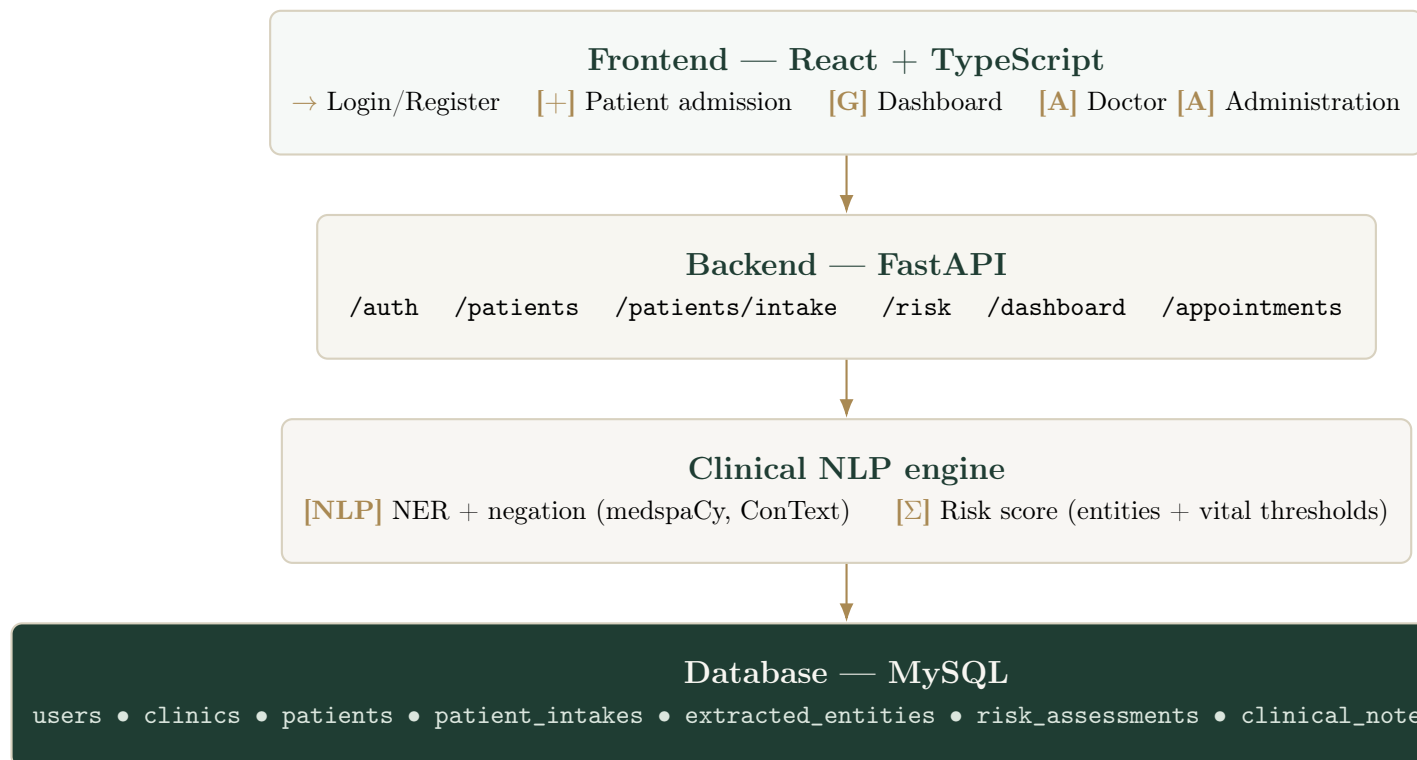


Figure 2.1: Layered architecture of the Smart Healthcare platform

2.1 Description of layers

2.1.1 Frontend

The frontend constitutes the platform's user interface. It exposes several main areas: authentication, patient admission, dashboard (patient and doctor), appointment management, notifications, account settings, support and administration. It is developed in React 18 with TypeScript, React Router for navigation, Axios for HTTP communication, and Tailwind CSS for the design system.

2.1.2 Backend

The backend is developed with FastAPI and exposes seven families of REST endpoints: /auth, /patients, /patients/intake, /risk, /dashboard, /appointments and /patients/{id}/notes. Each route is protected by a JWT authentication dependency and, where applicable, by role-based access control.

2.1.3 NLP engine

The NLP module extracts clinical entities from the entered text (symptoms, medical history, medications, allergies) and detects their negation before transmitting them to the risk engine.

It combines an English clinical pipeline based on *medspaCy* and a multilingual fallback engine (French, Arabic) based on a centralized clinical vocabulary.

2.1.4 Risk engine

The *Risk Engine* combines the retained NLP entities (non-negated, non-zero weight) and vital signs to produce a score, a risk level, and an explanation readable by the clinician.

2.1.5 Database

MySQL ensures persistent storage across eight main tables: `users`, `clinics`, `patients`, `patient_intakes`, `extracted_entities`, `risk_assessments`, `clinical_notes` and `appointments`, accessed via the SQLAlchemy ORM.

Implemented features

CHAPTER

This section details all the features actually developed in the project, module by module.

3.1 Authentication and account management

[key] Account lifecycle

- **Registration** (POST /auth/register) — account creation with automatic attachment to a clinic (joins an existing clinic by name, or creates a new one if it doesn't exist yet).
- **Login** (POST /auth/login and OAuth2 variant /auth/login/form) — returns a JWT (*Bearer token*), with account active status verification.
- **Current session** (GET /auth/me) — returns the authenticated user's profile from the token.
- **Profile update** (PATCH /auth/me) — full name and phone number.
- **Password change** (PATCH /auth/me/password) — requires verification of the current password before applying the new one.
- **Forgot password** (POST /auth/forgot-password) — generates a single-use reset token with limited lifetime, sent by email, with generic response (anti-email enumeration).
- **Reset** (POST /auth/reset-password) — consumes the token, verifies its expiration, updates the password and invalidates the token.

Password security. Passwords are hashed with `bcrypt` (secure truncation to 72 bytes before hashing). JWT tokens are signed with the HS256 algorithm and have a configurable lifetime.

3.2 Role and permission management

Three roles structure the entire platform:

Role	Main features
Patient	Profile management, creation of clinical admissions, consultation of their own risk results, appointment scheduling, consultation of their clinical notes, notifications.
Doctor	Access to all patients in their clinic, direct admission (walk-in), consultation and review of risk assessments, writing of clinical notes, appointment management.
Administrator	Overall view of the clinic (statistics), user account management (activation/deactivation), clinical audit log consultation, read-only access to all records.

Table 3.1: Role and permission matrix

Multi-tenant isolation

Each user and each patient is attached to a `clinic_id`. All read queries (patients, admissions, appointments, notes, statistics) systematically filter on the current user's clinic, ensuring that a doctor can never access data from another clinic.

3.3 Patient management

3.3.1 Patient profile (self-service)

A user with the *patient* role can create their own profile (POST `/patients`) with their administrative information: identity, gender, date of birth, blood group, height, weight, emergency contact. They can consult (GET `/patients/me`) and modify this profile.

3.3.2 Direct admission (walk-in)

A doctor or administrator can create a patient record **without an associated login account** (POST `/patients/intake/staff`) — identity and reason for consultation in a single request. This admission mode covers patients presenting physically to the clinic without having previously created an account. The record remains accessible only by the clinic's medical staff.

3.4 Clinical admission (*Patient Intake*)

Two admission pathways coexist and feed the same analysis pipeline:

1. **Patient admission** (POST `/patients/intake`) — the patient themselves provides reason for consultation, symptoms, medical history, current treatments, allergies and vital signs (temperature, blood pressure, heart rate, oxygen saturation).
2. **Healthcare staff admission** (POST `/patients/intake/staff`) — same clinical content, combined with patient identity creation.

Each submission immediately and synchronously triggers the analysis pipeline: NLP entity extraction, persistence of extracted entities, then risk score calculation.

3.5 Multilingual NLP pipeline

See details in Chapter 5.

3.6 Risk scoring engine

See details in Chapter 6.

3.7 Doctor dashboard

The doctor dashboard (GET `/dashboard/patients`) presents, in a single network round-trip, each admission with the patient's identity and their associated risk assessment, sorted by decreasing date. The view applies an **intelligent grouping by emergency contact**: multiple admissions sharing the same contact number are visually grouped and flagged as "to check," to help the clinician spot potential duplicate records for the same patient.

LOW**MODERATE****HIGH****CRITICAL**

Each risk level is associated with a consistent color code, used throughout the application (admissions list, appointments, dashboard) for immediate visual identification of priority cases.

3.8 Administrator dashboard

Reserved for the administrator role, this module centralizes:

- overall clinic statistics (`GET /dashboard/stats`): total number of patients, admissions, assessments, users, doctors, and distribution of admissions by risk level;
- user account management: role-filterable list, account activation/deactivation;
- clinical review audit log, tracking doctor decisions (confirmation or adjustment of AI score).

3.9 Clinical notes

Doctors and administrators can associate clinical notes with a patient record (`POST /patients/{patient_id}/notes`). These notes are timestamped, attributed to the author (doctor), and viewable by the patient themselves as well as by their clinic's staff (`GET /patients/{patient_id}/notes`), sorted from most recent to oldest.

3.10 Appointment management

The scheduling module allows:

- creation of an appointment (`POST /appointments`) by selecting a patient and a doctor from the clinic, a date/time and a reason;
- consultation of today's appointments, all statuses combined (`GET /appointments/today`);
- consultation of upcoming appointments (`GET /appointments/upcoming`);
- display of the patient's last known risk level directly on the appointment record, with color code, to visually prioritize sensitive consultations;
- on the patient side, consultation of their own appointments with status (scheduled, completed, cancelled, no-show).

3.11 Notifications

Each user has a notification center listing events concerning them (new appointments, new clinical notes), with:

- a filter "all / unread";
- individual or global marking ("mark all as read") as read;
- contextual redirection to the concerned resource on click.

3.12 Account settings

Each user can, from their "Settings" area, update their full name and phone number, as well as change their password (with verification of the current password and confirmation of the new one).

3.13 Support

A static support page groups a role-contextualized frequently asked questions section, as well as a direct contact link by email pre-filled with the connected account information (name, role, email address).

Patient pipeline architecture

CHAPTER

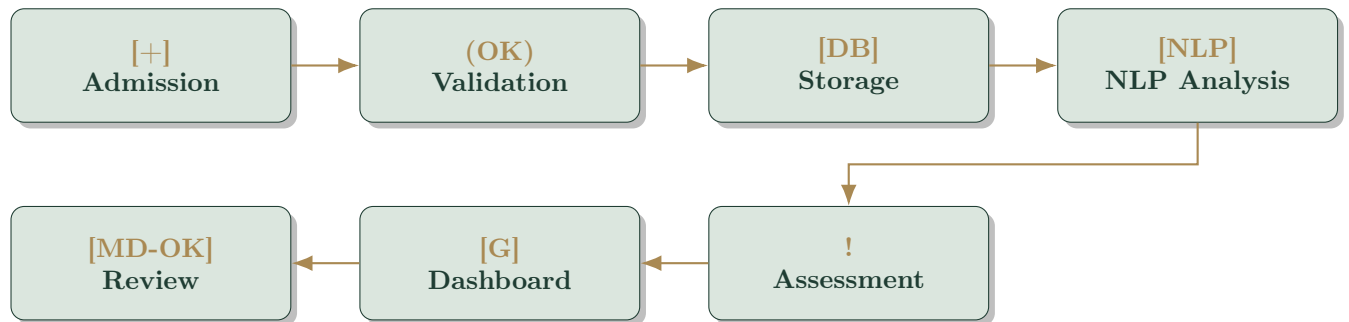


Figure 4.1: Complete patient admission pipeline

4.1 Pipeline steps

4.1.1 Admission

The patient (or healthcare staff in direct admission) provides the necessary information: identity (first name, last name, date of birth, gender), administrative information (blood group, height, weight, emergency contact), then clinical data: reason for consultation, symptoms, medical history, current medications, allergies, vital signs (temperature, blood pressure, heart rate, oxygen saturation) and additional notes.

4.1.2 Validation

The backend validates each field via Pydantic schemas: physiological bounds on temperature (35–42 °C), heart rate (20–250 bpm) and oxygen saturation (0–100 %), as well as the non-empty nature of required fields.

4.1.3 Storage

The data is saved in MySQL in the `patient_intakes` table, attached to the concerned patient.

4.1.4 Analysis

The clinical text (reason, symptoms, concatenated history) is transmitted to the NLP module, which returns the list of detected medical entities with their negation status.

4.1.5 Assessment

The retained entities (non-negated, with non-zero clinical weight) are combined with vital signs and transmitted to the risk engine, which calculates a score, a level and an explanation.

4.1.6 Presentation

The result is displayed to the doctor (and to the patient, for their own admissions) via the dashboard, with color code by risk level.

4.1.7 Review

The healthcare professional consults the complete record, can add a clinical note, and performs their own assessment, independently of the automatic score.

NLP architecture

CHAPTER

Natural language processing constitutes a central component of the platform. It relies on a **single source of truth**: a centralized clinical vocabulary, shared between the extraction module and the scoring engine, ensuring that the score truly depends on what the NLP extracted, without duplicated logic.

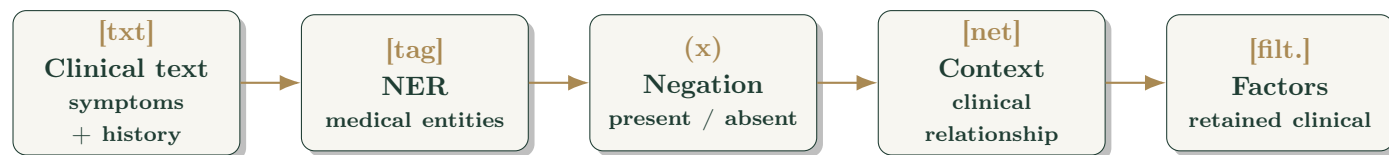


Figure 5.1: NLP extraction pipeline

5.1 Bilingual/multilingual architecture

Language	Extraction engine
English (en)	Clinical <i>medspaCy</i> pipeline: TargetRule (defined rules, without downloading a statistical model) and ConText (Chapman <i>et al.</i> , 2013) for negation detection.
French (fr)	Keyword search on the centralized clinical vocabulary, with negation detection by word window (cues: <i>pas de</i> , <i>aucun</i> , <i>sans</i> , <i>absence de</i> , <i>ne...pas</i> , etc.).
Arabic (ar)	Same keyword search and negation-by-window mechanism, with a dedicated cue lexicon (transliterated: " <i>lā yūjad</i> ", " <i>bidūn</i> ", " <i>yanfī</i> ", etc.).
Fallback	If <i>medspaCy</i> is unavailable (dependency not installed), the multilingual keyword engine automatically takes over, including for English.

Table 5.1: NLP extraction strategy by language

The text language is automatically detected (*langdetect*), with fallback to English in case of detection failure or missing dependency — the system must never block admission due to NLP unavailability.

5.2 Medical entity extraction

The clinical vocabulary covers four entity categories, each available in three languages:

- **Symptoms** — e.g., chest pain, shortness of breath, fever, cough, dizziness, fatigue.
- **History / diseases** — e.g., diabetes, hypertension, asthma, history of stroke.
- **Medications** — e.g., paracetamol, ibuprofen, insulin, metformin (zero weight, purely informational, excluded from score).
- **Allergies** — e.g., penicillin, peanuts, latex (zero weight, informational).

5.3 Negation detection

Negation is essential to avoid clinical interpretation errors.

Present	<i>"The patient presents with chest pain."</i>
Absent (negated)	<i>"The patient does not present with chest pain."</i>

The system distinguishes between the two situations before transmission to the risk engine: a negated entity is **retained in the extracted data** (for audit and traceability), but **excluded from score calculation**. The negation detection applies a search window bounded by proposition breaks (comma, "but", "however"), to avoid a negation in a distinct clause from incorrectly applying to a symptom mentioned later in the sentence.

5.4 NLP technologies

- Python 3.11
- spaCy
- medspaCy
- TargetRule
- ConText (Chapman *et al.*)
- langdetect
- Centralized clinical vocabulary (EN/FR/AR)
- Multilingual negation detection by window

Robustness

All imports of heavy NLP dependencies (`spacy`, `medspacy`, `langdetect`) are done defensively: the absence of any of these libraries should never prevent the server from starting or the admission pipeline from functioning.

Risk engine architecture

CHAPTER

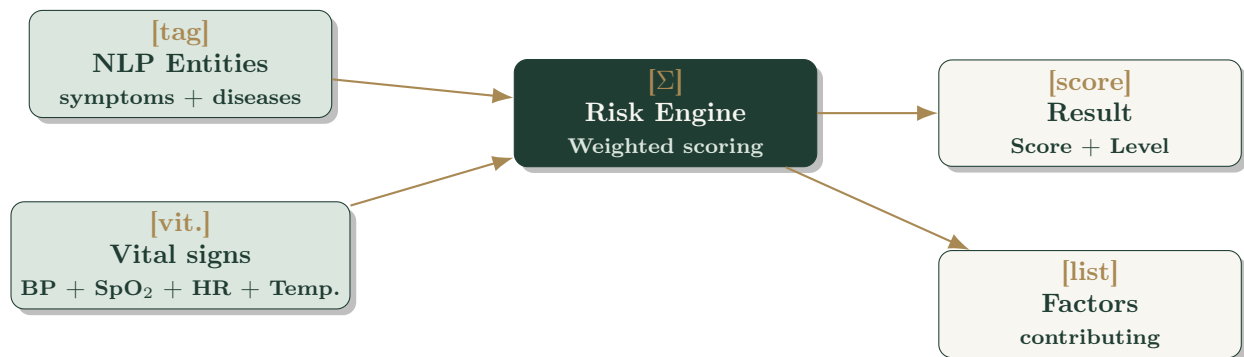


Figure 6.1: Risk scoring engine operation

6.1 Principle

The *Risk Engine* combines entities extracted from the text (already filtered by negation) with available vital signs. Each retained factor contributes to the score according to a weight defined in the centralized clinical vocabulary, ensuring total consistency between what the NLP detects and what the score takes into account.

6.2 Factors considered and weighting

Category	Examples	Weight
Critical symptoms	Chest pain, fainting, bleeding	25 – 35
Moderate symptoms	High fever, severe headache, confusion	20 – 25
Mild symptoms	Cough, headache, fatigue, nausea	3 – 10
History	Heart attack, stroke, pneumonia, cancer, covid	10 – 15
Chronic comorbidities	Diabetes, hypertension, asthma	5
Medications / allergies	Informational only	0

Table 6.1: Clinical entity weighting grid (excerpt)

Additional vital factors:

- High fever ($\geq 38.5^{\circ}\text{C}$): +20
- Abnormal heart rate (> 120 or < 50 bpm): +20
- Low oxygen saturation ($< 94\%$): +30
- High blood pressure ($\geq 160/100$): +20
- Low blood pressure ($< 90/60$): +15

The cumulative score is capped at 100. A **concept deduplication** mechanism avoids counting the same symptom twice if it is detected by both the *medspaCy* pipeline and the keyword search.

6.3 Risk levels

Level	Score threshold	Interpretation
LOW	< 15	Low risk
MODERATE	15 – 39	Moderate risk
HIGH	40 – 69	High risk
CRITICAL	≥ 70	Critical risk

6.4 Calculation example

For a patient presenting with: chest pain, shortness of breath, O₂ saturation of 91%, blood pressure of 165/100:

(i)

Risk Score: 100 / 100

• Risk Level: CRITICAL

• Model Confidence: 60%

The contributing factors are systematically returned as a readable list (for example: "*chest pain (+35); shortness of breath (+30); Low oxygen saturation 91% (+30); High blood pressure 165/100 (+20)*"), to make the result interpretable and verifiable by the healthcare professional.

Database architecture

CHAPTER

The platform uses MySQL with SQLAlchemy as the ORM. The schema is based on eight main entities, linked by foreign keys ensuring referential integrity and multi-clinic isolation.

7.1 Entity-relationship model

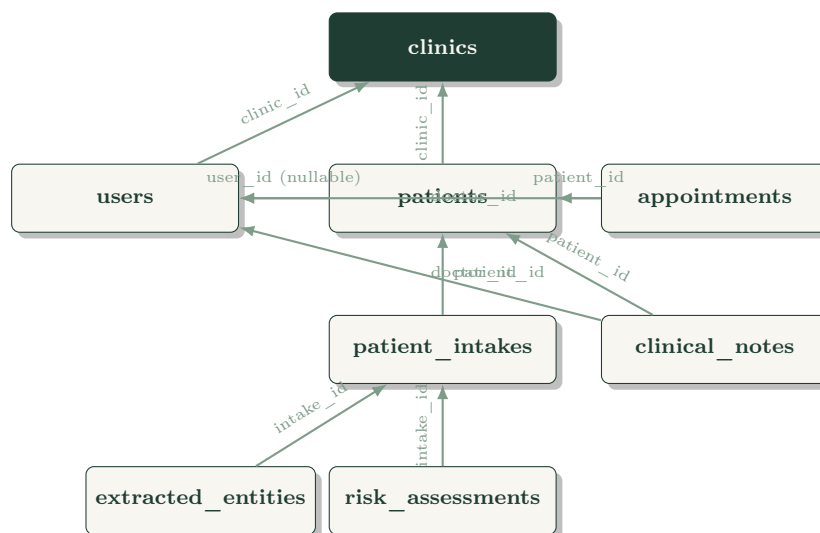


Figure 7.1: Simplified relational schema

7.2 Main entities

Table	Role and key fields
<code>clinics</code>	Healthcare establishment. <code>id</code> (UUID), <code>name</code> (unique).
<code>users</code>	User accounts (patient, doctor, admin). <code>id</code> , <code>full_name</code> , <code>email</code> (unique), <code>password_hash</code> , <code>role</code> , <code>clinic_id</code> , <code>phone</code> , <code>is_active</code> .
<code>patients</code>	Patient record (identity + administrative). <code>id</code> (UUID), <code>user_id</code> (nullable — direct admission), <code>created_by</code> , <code>clinic_id</code> , <code>identity</code> , <code>blood_group</code> , <code>height</code> , <code>weight</code> , emergency contact.
<code>patient_intakes</code>	Clinical admission. <code>id</code> (UUID), <code>patient_id</code> , <code>reason</code> , <code>symptoms</code> , <code>history</code> , <code>treatments</code> , <code>allergies</code> , <code>vital signs</code> , <code>status</code> .
<code>extracted_entities</code>	NLP entities detected for an admission. <code>intake_id</code> , <code>entity_type</code> , <code>entity_value</code> , <code>confidence</code> , <code>negated</code> (retained for audit, excluded from score if negated).
<code>risk_assessments</code>	Scoring result. <code>intake_id</code> , <code>risk_score</code> , <code>risk_level</code> (<i>enum</i> : low/medium/high/critical), <code>explanation</code> , <code>ai_confidence</code> , <code>reviewed_by</code> .

Table	Role and key fields
clinical_notes	Doctor observations. <code>patient_id</code> , <code>doctor_id</code> , <code>note</code> , <code>created_at</code> .
appointments	Scheduling. <code>patient_id</code> , <code>doctor_id</code> , <code>appointment_date</code> , <code>status</code> , <code>notes</code> (reason).

7.3 Relationships

Users are associated with patients and with various operations performed in the system (record creator, doctor for an admission or appointment, note author). Admissions are associated with patients. Extracted entities and risk assessments are associated with admissions. Clinical notes and appointments are associated with patients, with attribution to the concerned doctor.

Security and access control

CHAPTER

The platform uses a JWT-based authentication architecture and role-based access control (RBAC), combined with strict clinic-level isolation.

8.1 Authentication

The user authenticates via `/auth/login`. After password verification (bcrypt) and account active status check, the backend generates a signed JWT (HS256) containing the user identifier, with a configurable validity period. This token is then transmitted in the `Authorization: Bearer <token>` header of each request.

8.2 Authorization

Authentication alone is not sufficient. Each sensitive endpoint also verifies the user's role: direct admission routes, doctor dashboard and administration explicitly require a *doctor* or *administrator* role.

Defense in depth principle

The frontend should never be considered the sole security barrier. All permissions are verified on the backend side, regardless of what the interface allows or visually hides — a direct API request is systematically re-validated.

8.3 Data isolation

Each sensitive record (patient, admission, appointment, note) is attached to a clinic via `clinic_id`. All queries apply a systematic filter on the current user's clinic, preventing any cross-access between distinct clinics.

Frontend architecture

CHAPTER

9.1 Code organization

The frontend is structured according to a classic layered architecture for a data-oriented React application:

Layer	Role
<code>pages/</code>	Application screens (Login, Register, Dashboard, DoctorDashboard, AddPatient, Appointments, Notifications, Settings, Support, AdminPanel, RiskResult, PatientForm).
<code>components/</code>	Reusable components (Layout, ProtectedRoute, RoleHomeRedirect, VitalLine).
<code>contexts/</code>	Shared global state (AuthContext for session, ToastContext for transient notifications).
<code>services/</code>	API access layer: one module per resource (auth, patient, appointment, dashboard, user, notification), with conversion <i>snake_case</i> (API) → <i>camelCase</i> (frontend).
<code>types/</code>	TypeScript interfaces reflecting backend schemas.
<code>utils/</code>	Cross-cutting functions (e.g., home page resolution by role).

9.2 Routing and access protection

Routing (`react-router-dom`) associates each route with a set of authorized roles via a `ProtectedRoute` component: redirection to `/login` if not authenticated, redirection to the role's home page if access is denied. A `RoleHomeRedirect` component automatically routes the user to their dashboard (patient or doctor/admin) at the application root.

9.3 Session management

The `AuthContext` restores the session on application load: if a token is present locally, it is validated against `/auth/me`; in case of failure (expired or invalid token), the session is properly reset and the user redirected to login.

Frontend – Backend communication

CHAPTER

The frontend communicates with the backend through a REST API in JSON format.

React → Axios → FastAPI → Services → SQLAlchemy → MySQL

10.1 Endpoint mapping

Method	Route	Description
POST	/auth/register	Account creation
POST	/auth/login	Login, returns a JWT token
GET	/auth/me	Current user profile
PATCH	/auth/me	Profile update
PATCH	/auth/me/password	Password change
POST	/auth/forgot-password	Reset request
POST	/auth/reset-password	Reset via token
POST	/patients	Patient profile creation
GET	/patients/me	Current patient profile
GET	/patients	List of clinic patients
GET/PUT/DELETE	/patients/{id}	Consultation, modification, deletion
POST	/patients/intake	Admission (patient)
POST	/patients/intake/staff	Direct admission (doctor/admin)
GET	/patients/intake	List of accessible admissions
GET	/patients/intake/{id}	Admission details
GET	/patients/intake/{id}/entities	Extracted NLP entities
GET	/risk/{intake_id}	Risk assessment result
GET	/dashboard/patients	Enriched doctor view (identity + risk)
GET	/dashboard/stats	Clinic statistics
GET	/patients/{id}/notes	Patient clinical notes
POST	/patients/{id}/notes	Writing a clinical note
GET	/appointments/doctors	Clinic doctors
GET	/appointments/today	Today's appointments
GET	/appointments/upcoming	Upcoming appointments
POST	/appointments	Appointment creation

Technology stack

CHAPTER

Component	Technology
Frontend	React 18 + TypeScript
Routing	React Router
HTTP Client	Axios
UI	Tailwind CSS
Backend	FastAPI + Python 3.11
ORM	SQLAlchemy
Validation	Pydantic
Database	MySQL
Authentication	JWT (HS256) + bcrypt
ASGI Server	Uvicorn
NLP	spaCy + medspaCy
NER	TargetRule / centralized clinical vocabulary
Negation	ConText (medspaCy) + multilingual window
Language detection	langdetect
API	REST / JSON

Complete usage scenario (from A to Z)

CHAPTER

The complete system operation, from user login to final clinical review, proceeds according to the following steps.

1. The user logs in to the platform (or creates an account, attached to a clinic).
2. The system verifies their identity and generates a JWT token.
3. The system applies the permissions associated with their role on each request.
4. The patient completes their administrative profile (identity, emergency contact, etc.).
5. The patient provides the reason for consultation and describes their symptoms.
6. Medical history, current treatments and allergies are provided.
7. Vital signs are added if available.
8. The backend validates the information (physiological bounds, required fields).
9. The data is saved in MySQL (`patient_intakes`).
10. The clinical text is sent to the NLP module, with automatic language detection.
11. The NLP extracts medical entities (symptoms, history, medications, allergies).
12. The negation module determines, for each entity, whether it is present or negated.
13. Extracted entities are persisted (`extracted_entities`), including negated entities, for audit purposes.
14. The retained clinical factors (non-negated, non-zero weight) are transmitted to the Risk Engine.
15. The Risk Engine calculates the score by combining entities and vital signs.
16. The system determines the corresponding risk level (low/moderate/high/critical).
17. The contributing factors are recorded with their readable explanation.
18. The result is displayed in the doctor dashboard, with color code by risk level.
19. The dashboard automatically groups admissions by emergency contact to identify potential duplicate records.
20. The doctor consults the patient's complete record (admission, extracted entities, score).
21. The doctor can schedule a follow-up appointment, taking into account the displayed risk level.
22. The doctor can add a clinical note to the patient's record.
23. The patient is notified of events concerning them (appointments, new note).
24. The healthcare professional performs the final clinical review — the medical decision remains, under all circumstances, their sole responsibility.
25. The administrator has, in parallel, a consolidated view (statistics, account management, audit log) for overall clinic management.

Operation example

CHAPTER

13.1 Data entered

Reason	Chest pain and difficulty breathing
Symptoms	Recently appeared chest pain, shortness of breath and fatigue
History	Hypertension and type 2 diabetes
Temperature	37.2 °C
O ₂ Saturation	91%
Blood pressure	165/100

13.2 NLP extraction

Entity	Status	Weight
Chest pain	Present	+35
Shortness of breath	Present	+30
Fatigue	Present	+5
Diabetes	Present	+5
Hypertension	Present	+5

13.3 Assessment

The extracted information is combined with vital signs: low saturation ($< 94\%$, +30) and high blood pressure ($\geq 160/100$, +20).

(i)

Risk Score: 100 / 100 (capped) • **Risk Level:** **CRITICAL** • **Model Confidence:** 60%

13.4 Interpretation

The contributing factors are displayed to the doctor to allow immediate understanding of the result: "*chest pain (+35); shortness of breath (+30); fatigue (+5); diabetes (+5); hypertension (+5); Low oxygen saturation 91.0% (+30); High blood pressure 165/100 (+20)*". The doctor remains responsible for the final clinical interpretation and can plan immediate care given the displayed critical level.

Synthetic system view

CHAPTER



Figure 14.1: Complete processing chain, from admission to clinical review

Admission → Validation → NLP → Clinical extraction → Risk
Engine → Dashboard → Clinical review

Screenshots of interfaces

CHAPTER

This chapter presents all the interfaces actually implemented in the platform, organized by user group. Each section opens with a brief introduction recalling the functional scope covered by the following screens. All screenshots come from a local test environment (`localhost:5173`), with demo accounts and data.

15.1 Authentication

The authentication module covers the entire account lifecycle: creation, login, forgotten password recovery and reset via token sent by email. These screens are accessible without an active session and constitute the single entry point to the platform, regardless of the user's role.

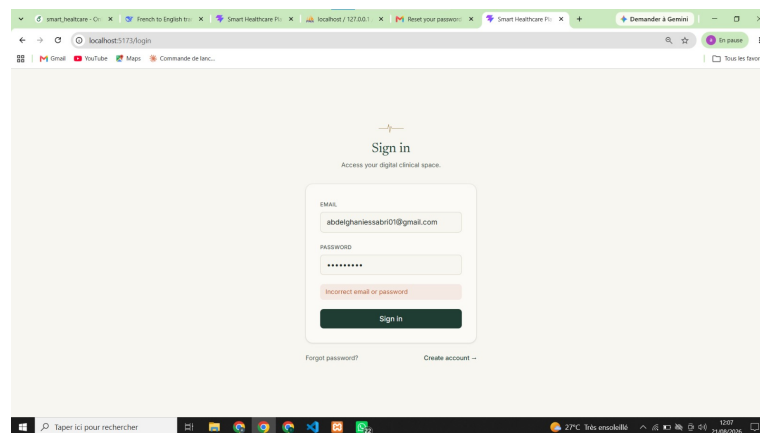


Figure 15.1: Login (`/login`) — with error message in case of incorrect credentials.

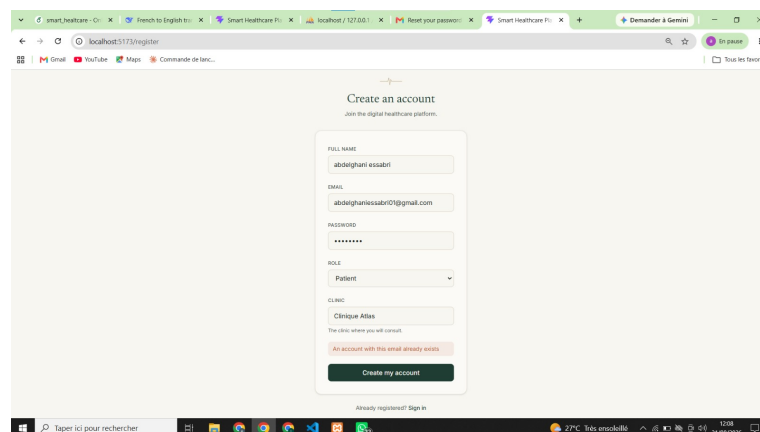


Figure 15.2: Account creation (`/register`) — role selection and clinic attachment.

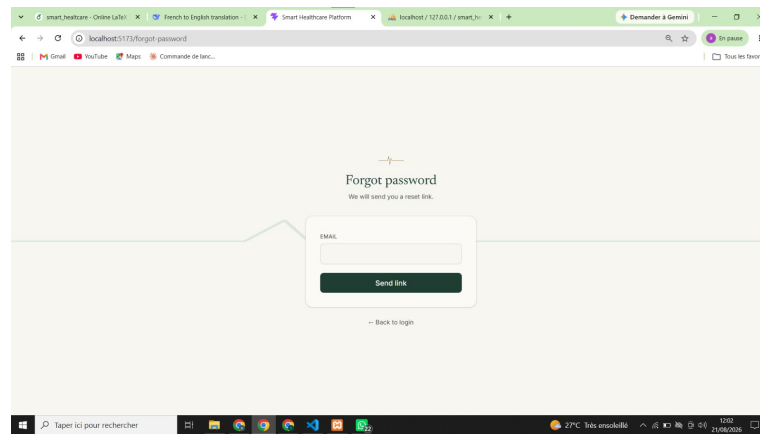


Figure 15.3: Forgot password (/forgot-password) — request a reset link.

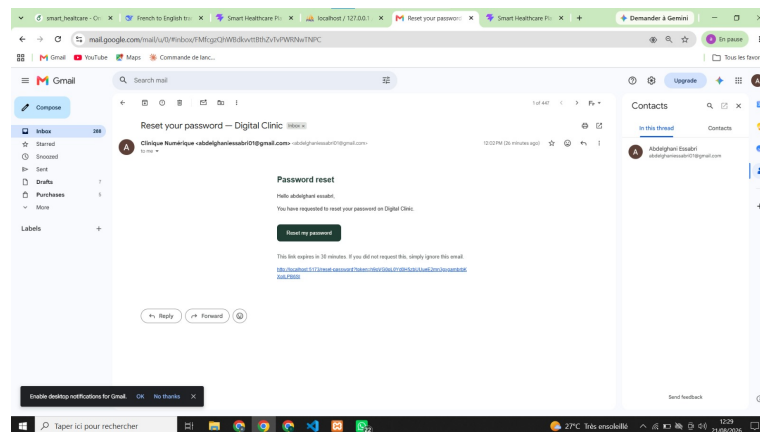


Figure 15.4: Reset email received, containing the link with limited lifetime (30 minutes).

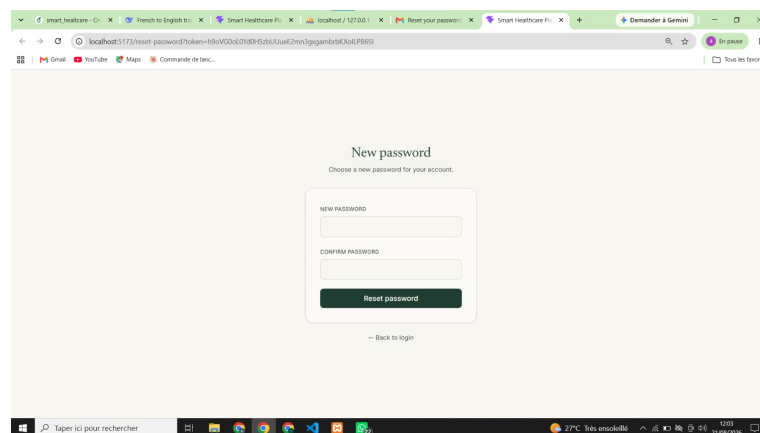


Figure 15.5: Password reset (/reset-password?token=...) — consuming the token received by email.

15.2 Patient Space

The patient space covers the self-service journey: viewing past admissions and their risk level, creating a new clinical admission, tracking appointments, viewing the detailed result of a risk assessment, and personal notification center.

15.2.1 Dashboard and admissions

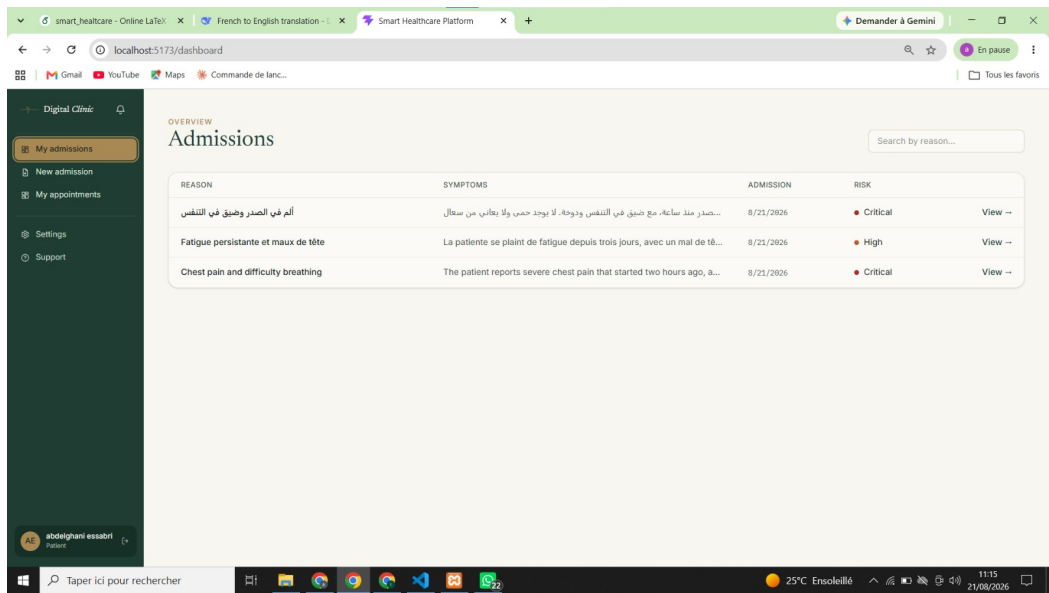


Figure 15.6: Patient dashboard (/dashboard) — admission history with risk level, including a case entered in Arabic.

15.2.2 New admission (/patient-form)

The admission form is presented in four progressive steps: patient profile, reason for consultation, medical history and treatments, then vital signs.

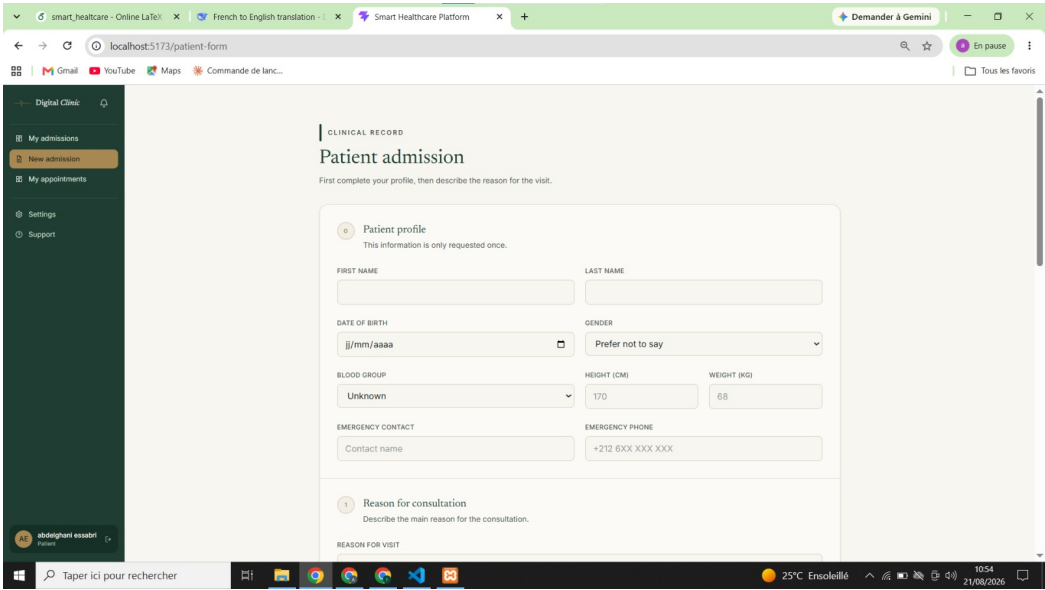


Figure 15.7: Step 0 — Patient profile (identity, date of birth, emergency contact).

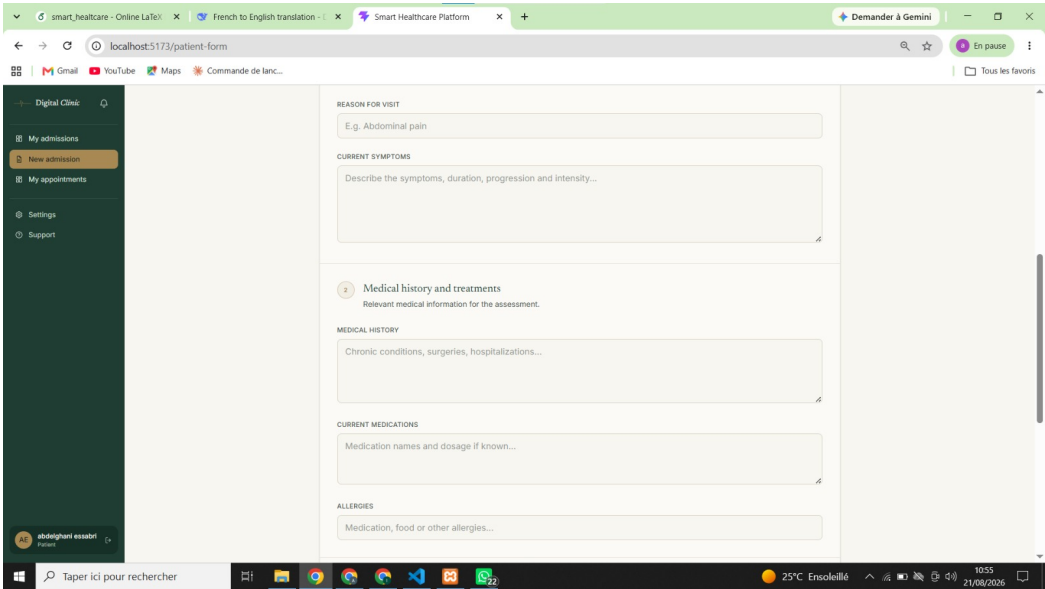


Figure 15.8: Steps 1 and 2 — Reason for consultation, current symptoms and medical history.

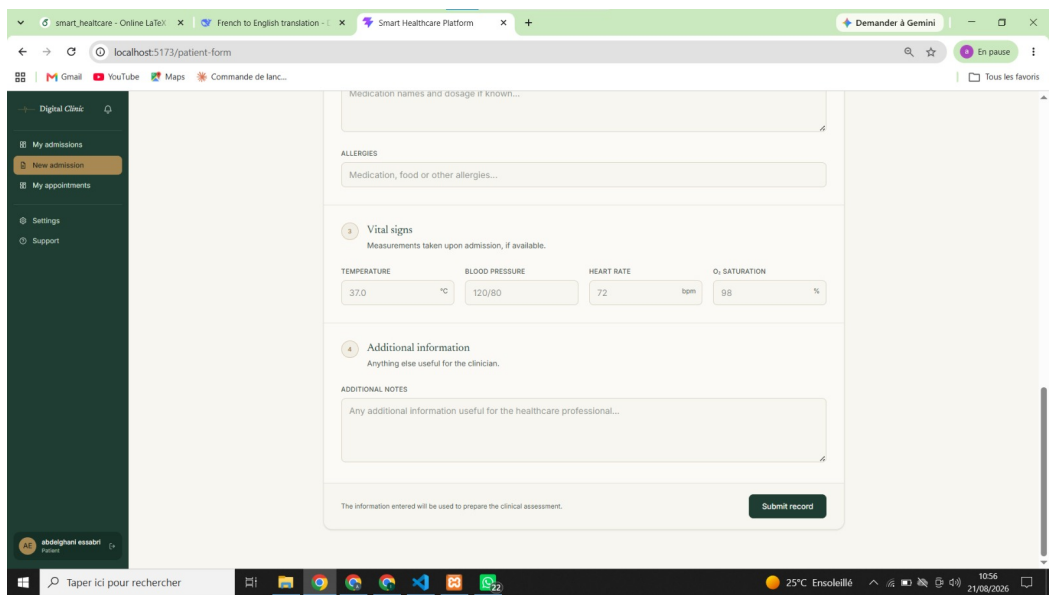


Figure 15.9: Steps 3 and 4 — Vital signs, additional information and form submission.

15.2.3 Risk assessment result

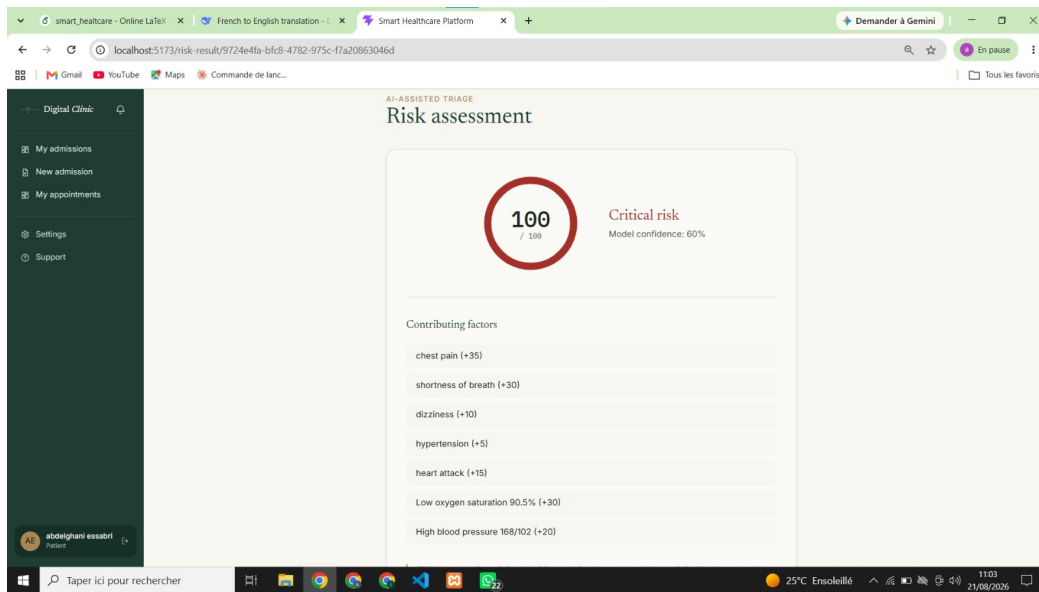


Figure 15.10: Risk assessment result (`/risk-result/{intake_id}`) — score, level and model confidence.

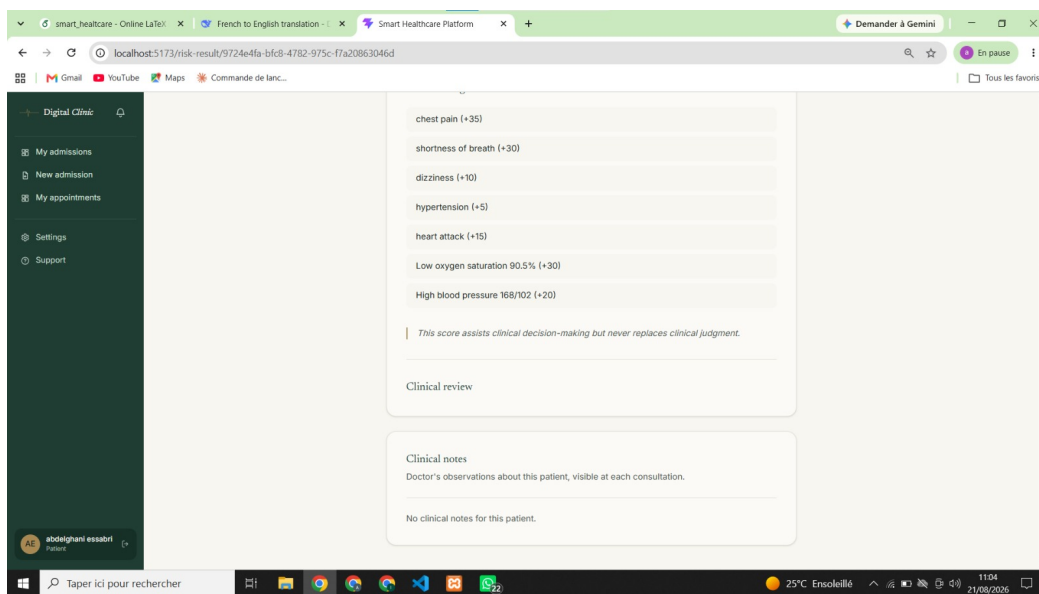


Figure 15.11: Detailed contributing factors, clinical disclaimer and medical notes section.

15.2.4 Notifications

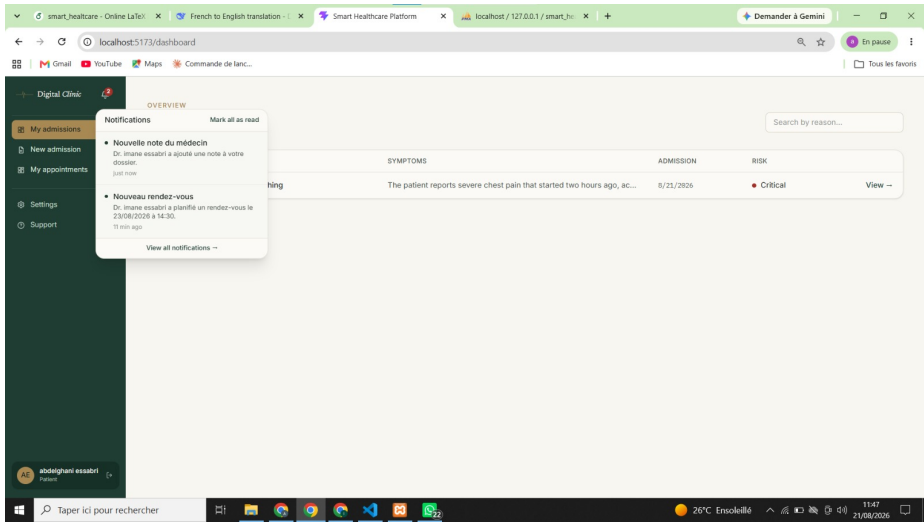


Figure 15.12: Quick preview of unread notifications, accessible from any page.

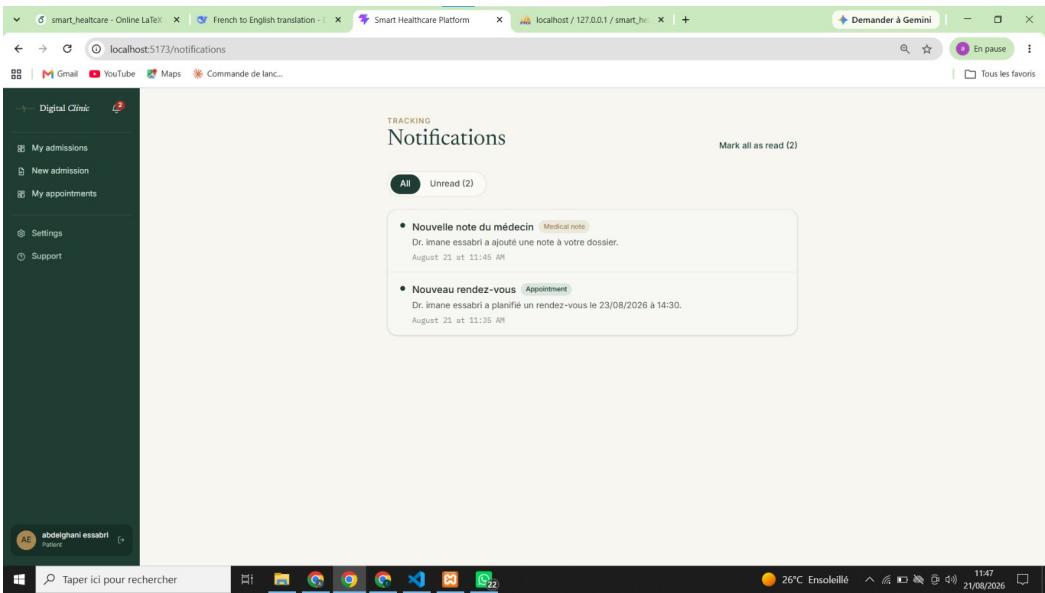


Figure 15.13: Full notification center (/notifications) — all/unread filter and global marking.

15.3 Doctor Space

The doctor space provides access to all patients in the clinic, with intelligent grouping by emergency contact to spot potentially duplicate records, as well as appointment scheduling.

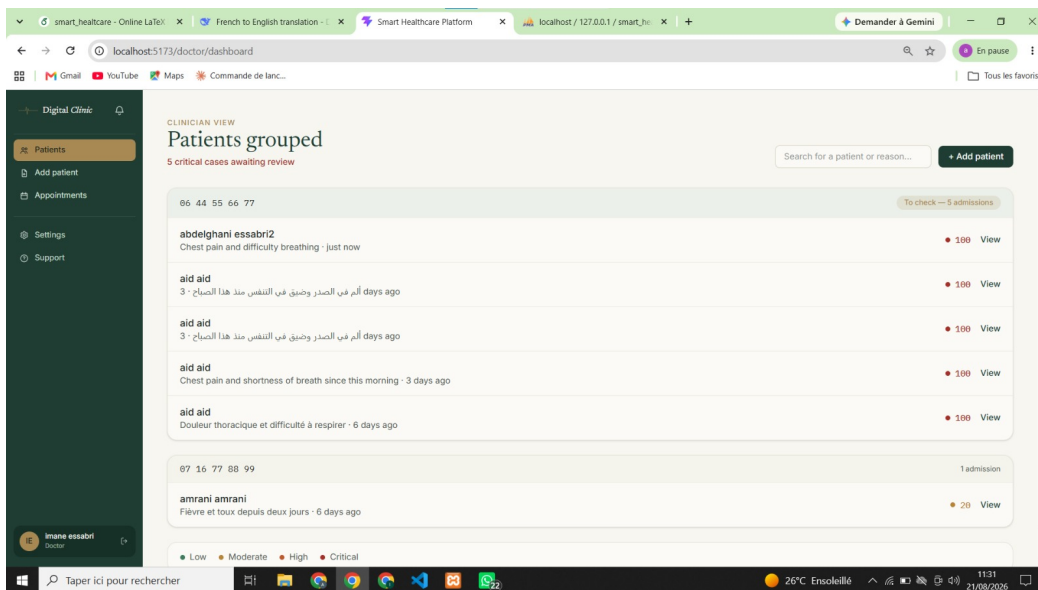


Figure 15.14: Clinician view (`/doctor/dashboard`) — patients grouped by contact, with risk level color coding.

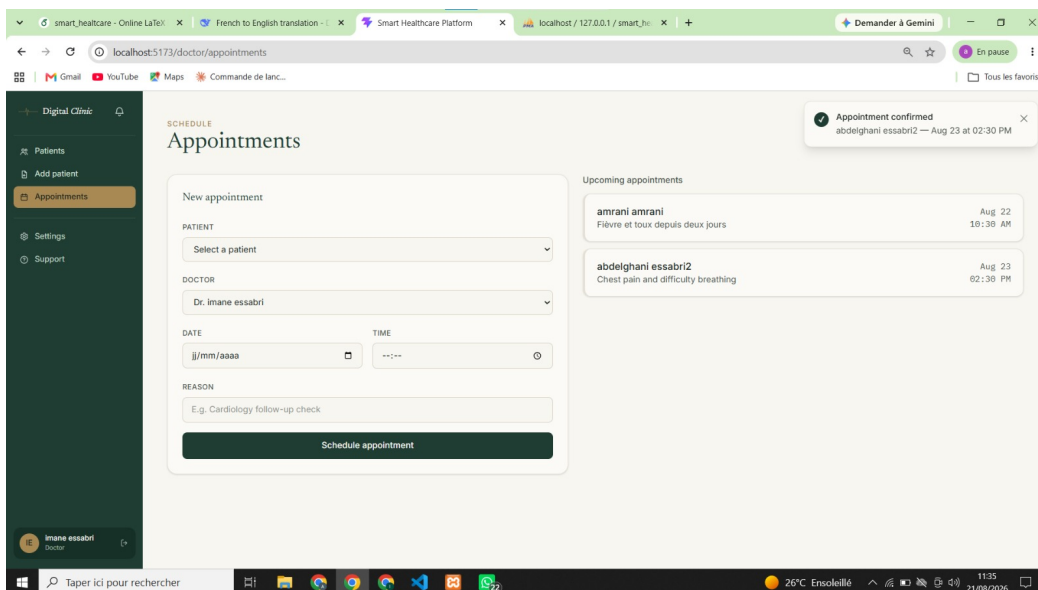


Figure 15.15: Appointment management (`/doctor/appointments`) — scheduling and list of upcoming appointments.

15.4 Administration

The administrator space offers a consolidated read-only view of the entire clinic: global statistics, user account management (activation/deactivation) and clinical review audit log.

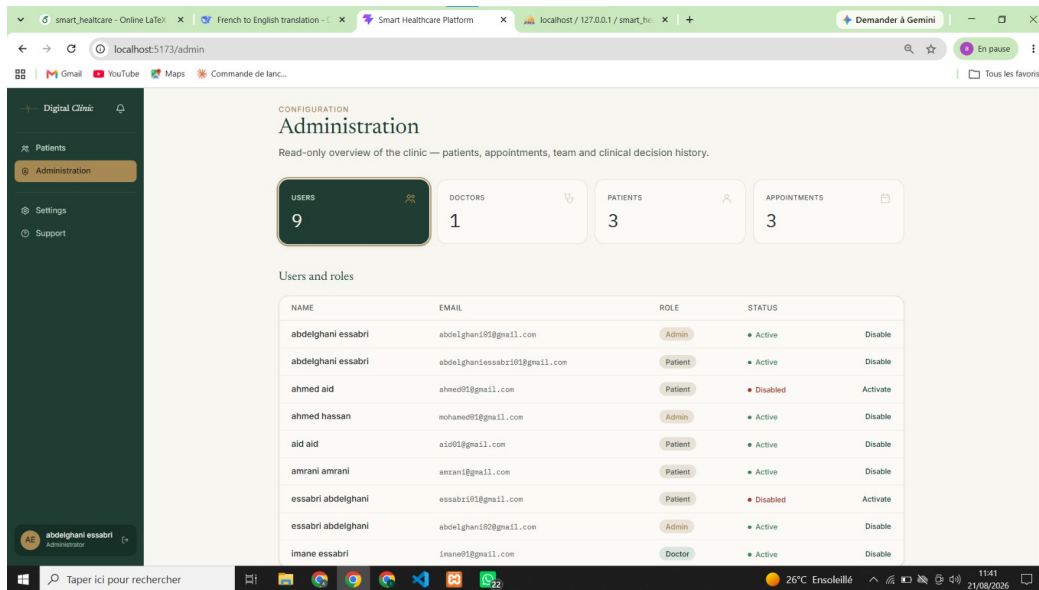


Figure 15.16: Administration (/admin) — clinic statistics and user list with their role and status.

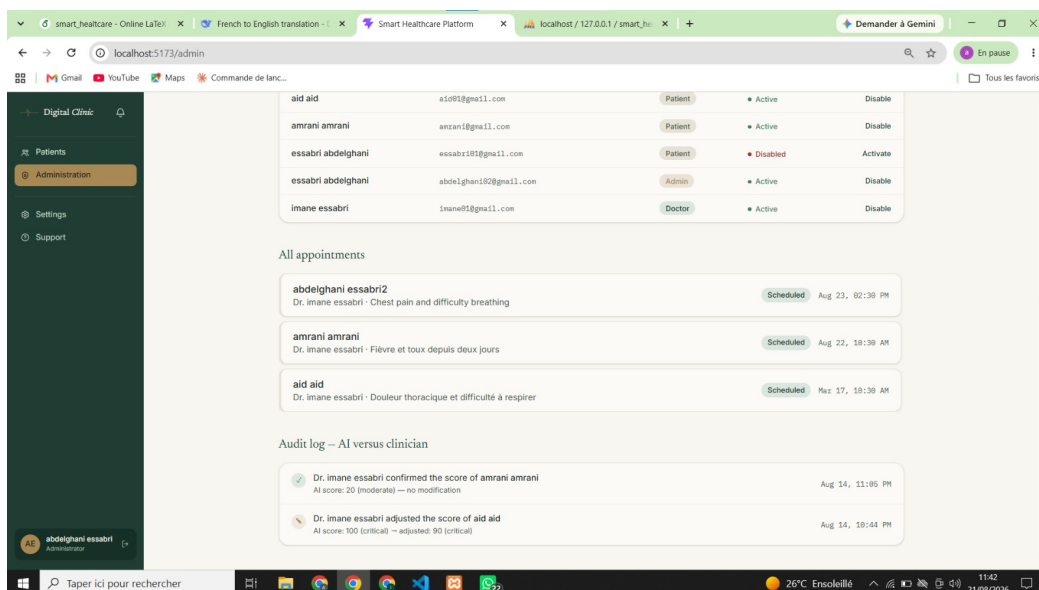


Figure 15.17: Administration page continuation — all appointments and AI versus clinician audit log.

Conclusion

CHAPTER

Smart Healthcare offers a complete architecture combining patient management, multilingual medical language processing, explainable risk assessment and clinical visualization. The system transforms medical information entered by the user into structured data, then into actionable indicators allowing the healthcare professional to better understand, prioritize and track their patient records.

The system's main chain can be summarized as follows:

**Admission → Validation → NLP → Clinical extraction → Risk Engine →
Dashboard → Clinical review**

(i)

Artificial intelligence provides analysis assistance. The final clinical decision always remains the responsibility of the healthcare professional.

Document written for the XelerondAI team — Smart Healthcare Platform, 2026.